

Bodega

Cómo está hecho el código: lenguajes, arquitectura y el render 3D

1. Los lenguajes y piezas grandes

frontend (JS+React+Three.js) --> HTTP/JSON --> backend (Python) --> base de datos
Vite empaqueta el JS REST API FastAPI + SQLAlchemy SQLite (dev) / Postgres (prod)

Backend: 100% Python, con FastAPI (framework web) + SQLAlchemy (ORM, traduce objetos Python a filas de base de datos) + JWT (tokens de sesión) + Pydantic (valida la forma de los datos que entran/salen de la API).

Frontend: JavaScript con sintaxis JSX (React), empaquetado con Vite, y Three.js para el 3D. Los estilos son CSS puro, escrito a mano, sin framework de UI. Es una app instalable (PWA): se agrega a la pantalla de inicio del celular y recibe avisos aunque esté cerrada.

Infraestructura: en producción, Render (el backend en Docker y el frontend como sitio estático) y Supabase (Postgres). En el computador, puertos propios: 5190 para la app y 8090 para la API. docker-compose.yml levanta los tres servicios (frontend, backend, Postgres) con un solo comando.

2. Backend — cómo está organizado

No es un solo archivo gigante, está separado por responsabilidad:

`backend/app/models.py`

Las 12 tablas — esto **es** la base de datos: users, room_config, elements (los muebles), products (cada código), stock (cuánto hay de cada código en cada ubicación), movements (el historial, con la nota de cada uno), documents (facturas, remisiones y conteos ya aplicados), parcels (lo de paso que no es inventario: cajas sueltas, canastas...), reserve_items (con su foto), code_aliases (etiquetas impresas con el código mal y el código bueno al que apuntan), push_subscriptions y app_settings. SQLAlchemy genera las tablas SQL a partir de estas clases Python.

`backend/app/layout_logic.py`

Funciones puras (sin base de datos) que traducen un mueble genérico ({type: "bins", cols: 9, rows: 8}) a sus ubicaciones concretas (C-1-1, C-1-2, ... 72 canastas), las 18 canastas debajo de la mesa (M-nivel-pila) y Despacho, la ubicación virtual de lo que está de paso. **Es el único lugar donde se calculan:** el visor 3D recibe cada ubicación con su nombre, y en la mesa con su nivel y su pila, y solo la dibuja. Así el nombre de cada ubicación es siempre el mismo en el backend y en el 3D. También marca qué ubicaciones son outlet: todo el mueble o lugares sueltos (por su lugar en el mueble, para que sigan siendo los mismos si cambia de tamaño).

backend/app/layout_service.py

Aplica cambios a la distribución (agregar/mover/borrar muebles, intercambiar letras) con una regla de negocio importante: **nunca deja una prenda con existencias sin ubicación válida**. Al cambiar una letra, cada ubicación se renombra por su lugar en el mueble, así las prendas se quedan en su canasta física. Por eso a veces el editor bloquea un cambio.

backend/app/inventory_service.py

Entradas/salidas/conteos. Un código puede estar en varias ubicaciones; una salida sin ubicación sale primero de Despacho, luego de la ubicación principal y después de donde haya. En Postgres usa `SELECT ... FOR UPDATE` para que si dos personas escanean el mismo código al mismo tiempo, la segunda espere en vez de pisar el dato de la primera (SQLite, el de desarrollo, escribe de a uno y lo ignora). También calcula qué pedir, sin pedir lo que ya está en la reserva, y traduce una etiqueta con el código mal al código bueno (`resolve_sku`) en cualquier movimiento.

backend/app/catalog.py

La conexión con la tienda. `pigmalionmoto.com` es Shopify, que publica el listado de productos en `/products.json` (sin claves, solo lectura). El servidor lo pide con `httpx` de a 250 productos por página, lo convierte en un diccionario por código `{sku: nombre, talla, precio, foto}` y lo guarda en memoria 6 horas. Lo lee en segundo plano al arrancar (`warm`), reintenta a los 90 segundos si falla y mientras tanto usa lo último que leyó. `near()` busca códigos casi iguales (una letra de diferencia y la misma talla al final) para las etiquetas impresas con el código mal. `/api/catalog/status` y `/refresh` muestran y renuevan la conexión.

backend/app/push.py

Avisos al celular (Web Push). Las llaves con que se firman se crean solas la primera vez y quedan en la base de datos: no hay nada que configurar en el servidor.

backend/app/migrations.py

Agrega columnas nuevas a una base que ya existe y hace cambios de una sola vez (por ejemplo, darle a la mesa sus 18 canastas en una distribución vieja).

backend/app/routers/

Los endpoints HTTP — cada archivo es un grupo de rutas: `/api/auth`, `/api/layout`, `/api/products`, `/api/movements`, `/api/reports` (con despachar), `/api/reserve` (escanear, enviar, despachar y devolver), `/api/documents` (factura, remisión y conteo), `/api/parcels` (lo de paso), `/api/catalog` (buscar, casi iguales, alias, estado) y `/api/push`.

backend/app/security.py, deps.py

Login con JWT y control de rol (administrador u operador). El interruptor `SKIP_AUTH` entra sin login, solo para desarrollo local; nunca en producción.

3. Frontend — cómo está organizado

pages/

Una página por pestaña — Warehouse (el 3D), Inventory, Scan, Reserve y Summary — más Count (conteo por ubicación), Reports y Login. React Router decide cuál mostrar según la URL; las direcciones viejas de Orders e History llevan al Resumen.

context/AuthContext.jsx

Guarda el token JWT y el usuario logueado, disponible para toda la app sin pasarlo manualmente por cada componente (patrón "Context" de React).

`hooks/useApi.js`

En vez de WebSockets, usa sondeo (polling) con una sola caché por ruta para toda la app: el inventario se pide cada 6 segundos, los movimientos y la reserva cada 8, lo que hay que reponer cada 10, lo de paso cada 15 a 20 y los reportes cada 30 segundos o más. Las acciones (+1, -1, guardar) cambian la pantalla en el acto y el servidor confirma por detrás. Es una decisión consciente de simplicidad.

`hooks/useBarcodeScanner.js`, `lib/barcode.js`

El lector de códigos con la cámara: el lector nativo del celular en Android y ZXing (compilado a WebAssembly) en iPhone, con linterna, lectura desde una foto, y la cámara se apaga sola si hay un formulario encima, si la app pasa a segundo plano o tras 45 segundos sin leer.

`lib/ocr.js`, `lib/facturaParser.js`

Lee la factura impresa con Tesseract.js en el mismo celular (la foto no sale del teléfono) y corrige cada línea contra los códigos conocidos antes de mostrarla para revisar.

`components/`

Piezas de UI reutilizables: las hojas que se arrastran desde abajo (Sheet), EditPanel, LocationSheet, ProductModal, RemisionSheet, FacturaSheet, ParcelSheet, etc. Cada una habla con la API mediante `api.js`.

`public/sw.js`

El service worker: hace la app instalable y muestra los avisos que llegan aunque esté cerrada.

4. El render 3D — la parte más interesante

Vive en un solo archivo, `frontend/src/three/WarehouseScene.js`, y es una **clase de JavaScript aislada de React** — React solo le pasa datos (`setLayout`, `setProducts`) y escucha eventos de toque. Nada de HTML/CSS ahí adentro, todo es Three.js puro. Los ids y nombres de cada ubicación llegan siempre del backend; el 3D nunca los recalcula.

No hay modelos 3D importados (ni .glb, ni .obj, nada descargado). Cada mueble se construye por código con formas geométricas primitivas de Three.js:

```
// una estanteria es, literalmente, codigo que arma cajas:
const post = new THREE.BoxGeometry(0.06, H, 0.06) // las 4 patas
const beam = new THREE.BoxGeometry(W, 0.08, 0.05) // las vigas
const deck = new THREE.BoxGeometry(W - 0.04, 0.02, D - 0.04) // el nivel
```

Por eso cuando agregas una estantería de 5 niveles en vez de 3, no carga un archivo distinto — el mismo código matemático genera 5 niveles en vez de 3.

Piezas clave del motor

1. Construcción desde datos (`_buildWorld`)

Recibe el JSON del backend (`layout.elements`) y por cada mueble llama a su función (`_buildShelf`, `_buildRack`, `_buildBins`, `_buildTable`...) que arma la geometría. Las canastas de la mesa se dibujan con el nivel y la pila que manda el backend. Si el mundo se reconstruye (por ejemplo al agregar un mueble), Three.js libera la memoria de las formas viejas (`geometry.dispose()`) para no dejar fugas.

2. Instancing para rendimiento

Una pared de canastas puede tener 72 gavetas — en vez de crear 72 objetos 3D separados (lento), usa `THREE.InstancedMesh`: una sola geometría dibujada 72 veces con distintas posiciones, en una sola llamada a la GPU. Es la técnica estándar para repetir miles de objetos iguales sin matar el rendimiento en un celular.

3. Cámara orbital manual

No usa el OrbitControls de ejemplo de Three.js — está escrita a mano con coordenadas esféricas (th = ángulo horizontal, ph = ángulo vertical, r = distancia) que se animan suavemente hacia un "objetivo" (goal) cada cuadro. Así los botones "General" / "Planta" / enfocar un mueble son solo "cambia el objetivo de la cámara" y se desliza solo. Se mueve con los dedos: arrastrar para girar y pellizcar para acercar.

4. Detección de toques (raycasting)

Cada ubicación y cada mueble tiene una caja invisible (hitMat, opacidad 0). Cuando tocas la pantalla, se lanza un rayo desde la cámara hasta ese punto y se pregunta "¿con qué caja invisible chocó?" — así sabe qué ubicación o mueble tocaste, sin depender de coordenadas de pantalla.

5. Realismo: look "estudio"

Materiales PBR (MeshStandardMaterial) con roughness/metalness para que la luz rebote de forma creíble (metal vs. cartón vs. concreto), y geometría con **bordes redondeados** (RoundedBoxGeometry).

Texturas procedurales generadas en código, no descargadas: se dibuja en un <canvas> invisible y ese canvas se usa como textura — así el piso tiene moteado de concreto y las cajas tienen líneas de cartón corrugado + cinta, sin bajar ninguna imagen de internet.

Luz: luz de entorno (RoomEnvironment), una luz principal cálida, un relleno frío y una luz hemisférica. **Sombras suaves** (PCFSoftShadowMap) que se recalculan solo cuando algo cambia en la bodega, no en cada cuadro.

Tono AgX (el mismo de Blender) y **oclusión ambiental GTAO**: sombras de contacto en los rincones y entre las canastas, en vez de la niebla de la primera versión. El cuarto es una maqueta en corte: las paredes que dan a la cámara bajan solas.

5. Las reglas del negocio

Lo que hace la app, en pocas palabras:

Una prenda en varias ubicaciones (tabla stock), con traslados entre ellas y conteo por ubicación que corrige solo esa ubicación.

Remisión (lo que llega del proveedor): se cuenta por talla a mano, con la foto solo a la vista y notas; va a la bodega, a la reserva o de paso. El mismo número no entra dos veces.

Factura (lo que salió en cajas): foto, lectura en el celular, revisión y se descuenta todo junto o nada.

De paso: lo que entra solo para despacharse no es de la bodega. Las prendas con código quedan en Despacho y salen primero; las cajas sueltas, canastas o bolsas se anotan con de quién son y qué hacer, y salen con "Ya salió". Nada de esto cuenta para pedidos, reportes ni el valor del inventario.

Reserva: mercancía guardada aparte. El Inventario muestra bodega + reserva = total por referencia y por talla, y el Resumen avisa qué traer de la reserva antes de pedir.

Avisos al celular de entradas, salidas, conteos, bajo mínimo y documentos. A quien hizo el movimiento no le llega su propio aviso, salvo el de bajo mínimo, que le llega a todos.

Outlet: prendas ubicadas en la bodega que no cuentan: ni en lo que hay, ni en pedidos, ni en el valor. Una salida sin ubicación nunca las toca; se sacan eligiendo esa ubicación.

De dónde sale: una salida (escaneo o factura) de una prenda que está en varios lugares pregunta de cuál; sale primero de ahí y, si no alcanza, el resto de donde haya.

Despacho: lo que sale empacado se despacha con una nota y sale del inventario; se puede despachar directo desde la reserva. Lo de paso espera en Despacho y lo que sobra vuelve a la reserva. Pasar entre bodega y reserva no cuenta como entrada ni salida en los reportes.

Sin contar dos veces: al registrar en la bodega algo que está en la reserva, se pregunta si viene de allá y se descuenta de la reserva. Reserva y bodega se comparan por código o por referencia y talla, sin importar tildes.

Cuidado con los datos: lo escaneado puede quedar en una lista por confirmar (guardada en el celular), la cámara puede leer una prenda a la vez, y todo lo que borra pide confirmación.

6. En resumen

Es una arquitectura bastante estándar y sin trucos raros: API REST clásica + base de datos relacional + un motor 3D casero hecho de formas geométricas simples, no de modelos importados. La complejidad real no está en gráficos avanzados, sino en la lógica de negocio: **no perder inventario, no chocar códigos, no pisar datos entre dos personas escaneando a la vez, y no mezclar lo que está de paso con lo que es de la bodega.**

Actualizado el 6 de octubre de 2026 a partir del mapa del código (graphify) y del código actual.